

Teste de Software

Detecção de Bad Smells e Refatoração Segura

João Pedro Silva Braga
815051

Belo Horizonte, MG
9 de novembro de 2025

1 Análise de Débitos Técnicos no Código Original

O arquivo de produção `src/ReportGenerator.js` foi submetido a uma análise manual detalhada para identificar "Bad Smells" sintomas de problemas estruturais que comprometem a qualidade do software. A análise revelou que a função principal, `generateReport`, sofria de vários débitos técnicos que dificultavam sua manutenção, extensibilidade e testabilidade. A seguir, detalhamos três dos smells mais críticos identificados.

1.1 Método Longo e Monolítico (Long Method)

A função `generateReport` era excessivamente longa, centralizando múltiplas lógicas de negócio distintas em um único bloco de código. Ela era responsável simultaneamente pela filtragem de dados baseada no perfil do usuário, pela lógica de priorização de itens, pela formatação da saída em múltiplos formatos (HTML e CSV) e pelo cálculo de valores agregados.

Um método com essa estrutura monolítica é inerentemente frágil. A alta coesão acidental torna qualquer alteração arriscada, pois uma mudança em uma regra de negócio (ex: lógica de priorização) pode introduzir regressões inesperadas na formatação do relatório. Além disso, a complexidade do código dificulta o entendimento e torna a escrita de testes unitários granulares e focados uma tarefa quase impossível.

1.2 Violação do Princípio da Responsabilidade Única (SRP)

Como consequência direta do smell anterior, a classe `ReportGenerator` violava o Princípio da Responsabilidade Única (SRP). Ela acumulava responsabilidades que deveriam ser delegadas a componentes distintos, como a política de acesso (quem pode ver o quê), a lógica de formatação de dados e a execução de cálculos.

A violação do SRP resulta em um alto acoplamento entre conceitos não relacionados. Por exemplo, a lógica de apresentação (HTML/CSV) estava diretamente acoplada à lógica de negócio (regras de acesso). Isso torna o sistema rígido e difícil de evoluir. Adicionar um novo formato de relatório (ex: PDF) ou uma nova regra de acesso exigiria modificar o mesmo método extenso, aumentando o risco de erros.

1.3 Mutações e Efeitos Colaterais (Side Effects)

Durante o processamento, o código original modificava diretamente os objetos de entrada. Especificamente, itens com valor superior a 1000 recebiam a adição de uma nova propriedade `priority`. Essa prática, conhecida como mutação de estado, introduz efeitos colaterais (side effects).

Funções com efeitos colaterais são imprevisíveis e difíceis de depurar. Se o array de `items` fosse reutilizado em outra parte do sistema após a chamada de `generateReport`, ele conteria dados modificados, podendo levar a comportamentos inesperados e bugs difíceis

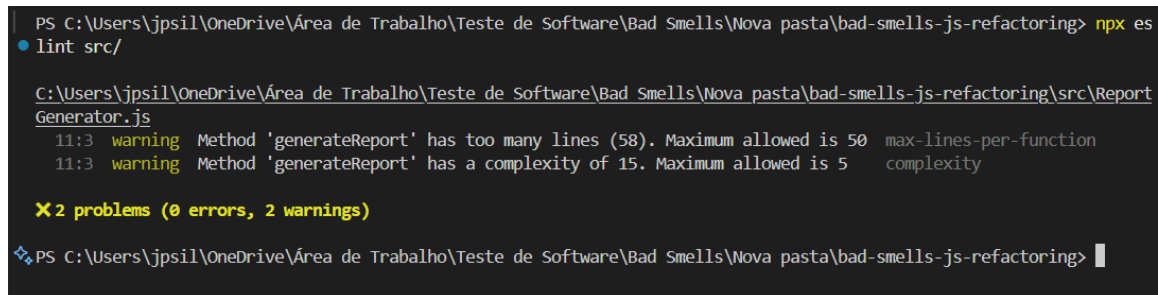
de rastrear. A imutabilidade, por outro lado, garante que as funções sejam "puras" e determinísticas, facilitando os testes e o raciocínio sobre o fluxo de dados.

2 Detecção Automatizada com ESLint e SonarJS

Para complementar a análise manual e obter métricas objetivas, foi configurada uma ferramenta de análise estática utilizando ESLint em conjunto com o plugin `eslint-plugin-sonarjs`. Este plugin é especializado na detecção de "code smells" e problemas de manutenibilidade.

2.1 Resultados da Análise Estática

A execução do linter sobre o código original (`npx eslint src/`) confirmou as observações manuais e revelou problemas quantitativos que poderiam passar despercebidos, como a alta complexidade cognitiva.



```
PS C:\Users\jpsil\OneDrive\Área de Trabalho\Teste de Software\Bad Smells\Nova pasta\bad-smells-js-refactoring> npx es
• lint src/

C:\Users\jpsil\OneDrive\Área de Trabalho\Teste de Software\Bad Smells\Nova pasta\bad-smells-js-refactoring\src\Report
Generator.js
  11:3  warning  Method 'generateReport' has too many lines (58). Maximum allowed is 50  max-lines-per-function
  11:3  warning  Method 'generateReport' has a complexity of 15. Maximum allowed is 5    complexity

X2 problems (0 errors, 2 warnings)
❖ PS C:\Users\jpsil\OneDrive\Área de Trabalho\Teste de Software\Bad Smells\Nova pasta\bad-smells-js-refactoring>
```

Figura 1: Relatório do ESLint e SonarJS apontando alta complexidade e um número excessivo de linhas no arquivo `ReportGenerator.js` antes da refatoração.

2.2 Análise Crítica da Ferramenta

O plugin `sonarjs` foi fundamental para a análise, atuando como uma "segunda opinião" objetiva e automatizada. Enquanto a análise manual identificou o "Método Longo" de forma qualitativa, o SonarJS quantificou o problema através da métrica de **complexidade cognitiva**. Um valor elevado nesta métrica indica que o código possui muitas ramificações (ifs, loops, switches) aninhadas, tornando-o difícil de ser compreendido e modificado por um desenvolvedor. A ferramenta também apontou duplicações de código e outros padrões que reforçaram a necessidade urgente de refatoração, fornecendo um direcionamento claro para as melhorias.

3 Processo de Refatoração Segura

O smell mais crítico a ser atacado foi o **Método Monolítico**, pois ele era a causa raiz dos outros problemas, como a violação do SRP e a complexidade elevada. A estratégia

foi decompor a função original em unidades menores, coesas e com responsabilidades bem definidas.

3.1 Código Original (Antes)

O trecho abaixo ilustra a complexidade e o acoplamento do método original, onde a lógica de acesso, mutação e formatação estão entrelaçadas.

```
1 // Trecho representativo (versão original)
2 for (const item of items) {
3   if (user.role === 'ADMIN') {
4     if (item.value > 1000) {
5       item.priority = true; // Mutação de dados de entrada
6     }
7     // ... lógica de formatação para CSV/HTML misturada aqui
8   } else if (user.role === 'USER') {
9     if (item.value <= 500) {
10      // ... lógica de formatação duplicada
11    }
12  }
13 }
```

Listing 1: Trecho do método ‘generateReport’ original, mostrando aninhamento e mutação.

3.2 Código Refatorado (Depois)

A solução foi aplicar a técnica **Extract Method** para isolar a política de acesso em uma função pura e independente. Esta nova função não possui efeitos colaterais e retorna um novo array, preservando a imutabilidade.

```
1 function applyAccessPolicy(user, items) {
2   if (user?.role === 'ADMIN') {
3     // Retorna um novo array sem modificar o original
4     return items.map(it => ({ ...it, priority: it.value > 1000 }));
5   }
6   if (user?.role === 'USER') {
7     return items.filter(it => it.value <= 500);
8   }
9   return []; // Retorno explícito para outros casos
10 }
```

Listing 2: Função extraída para gerenciar a política de acesso de forma pura e imutável.

3.3 Técnicas de Refatoração Aplicadas

O processo de melhoria foi guiado por um conjunto de técnicas e princípios de design, garantindo uma refatoração segura e eficaz:

- **Extract Method:** Decomposição da função monolítica em funções menores e com propósito único (ex: `applyAccessPolicy`, `formatToCsv`, `calculateTotal`).
- **Princípio da Responsabilidade Única (SRP):** Atribuição de responsabilidades claras a cada nova função, separando lógica de acesso da lógica de formatação.
- **Imutabilidade:** Utilização do operador *spread* (`...it`) e de métodos como `map` e `filter` para evitar a mutação de dados de entrada, resultando em funções puras e previsíveis.
- **Decompose Conditional:** Simplificação de blocos `if/else` aninhados, tornando o fluxo de controle mais claro e linear.

4 Conclusão

Este trabalho demonstrou na prática a importância da detecção e correção de Bad Smells para a saúde de um projeto de software. A refatoração guiada por princípios de design e apoiada por ferramentas de análise estática transformou um código frágil e complexo em um sistema modular, legível e seguro para evoluir.

A suíte de testes existente provou ser um ativo inestimável, atuando como uma **rede de segurança** que permitiu realizar alterações estruturais profundas com a confiança de que o comportamento externo do sistema não seria quebrado. A cada passo da refatoração, a execução dos testes garantia que nenhuma regressão havia sido introduzida.

Ao final, a redução drástica dos Bad Smells não apenas eliminou os alertas do linter, mas resultou em um código com menor complexidade cognitiva, maior coesão e menor acoplamento. Isso se traduz diretamente em benefícios de longo prazo: manutenções futuras serão mais rápidas e seguras, a adição de novas funcionalidades (como outros formatos de relatório) será trivial e a base de código se torna um ativo, em vez de um passivo técnico. A combinação de testes automatizados e análise estática cria, de fato, um ciclo virtuoso de melhoria contínua da qualidade de software.